MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Asynchronous Communication and Real-Time Notification

Student: Bode Andreea-Nicoleta

Profespr: Chis David

Facultatea de Automatica si Calculatoare

Specializarea: Tehnologia Informatiei

Cuprins

1. Cerinta 3
2. Implementare 4
3. Readme ..... **Eroare! Marcaj în document nedefinit.**
Frontend 8
Device, User Microservices, Producer si Consumer 9
4. Bibliografie 9

1. Cerinta

Implement a Monitoring and Communication Microservice for the Energy Management System. The microservice is based on a message broker middleware that gathers data from the smart metering devices, processes the data to compute the hourly energy consumption and stores it in the database of the Monitoring and Communication Microservice. The synchronization between the databases of Device Management Microservice and the new Monitoring and Communication Microservice is made through an event-based system that uses a topic for device changes (sends device information through a queue for the Monitoring and Communication Microservice). A Smart Metering Device Simulator application will be implemented as the Message Producer. It will simulate a smart meter by reading energy data from a sensor.csv file (i.e., one value at every 10 minutes) and sends data in the form < timestamp, device_id, measurement_value > to the Message Broker (i.e., a queue). The timestamp is taken from the local clock, and the device_id is unique to each instance of the Smart Metering Device Simulator and corresponds to the device_id of a user from the database (as defined in Assignment 1). The device simulator should be developed as a standalone application (i.e., desktop application)

Frontend

- Interfața grafică a utilizatorului (UI) care permite autentificarea utilizatorilor, gestionarea conturilor și dispozitivelor inteligente, vizionarea graficelor si a notificarii.

Microserviciu pentru Managementul Utilizatorilor

- Gestionarea conturilor de utilizator: adăugare, citire, actualizare și ștergere (CRUD).
- Autentificarea și autorizarea utilizatorilor.
- Crearea și gestionarea mapărilor dintre utilizatori și dispozitive.

Microserviciu pentru Managementul Dispozitivelor

- Gestionarea dispozitivelor inteligente: adăugare, citire, actualizare și ștergere (CRUD).
- Limitarea consumului maxim de energie pe oră pentru fiecare dispozitiv.
- Asocierea dispozitivelor cu utilizatorii corespunzători.
- Este responsabil pentru trimiterea deviceId-ul si a valorii maxime prin cozi spre Consumer.

Microserviciu pentru Consumer

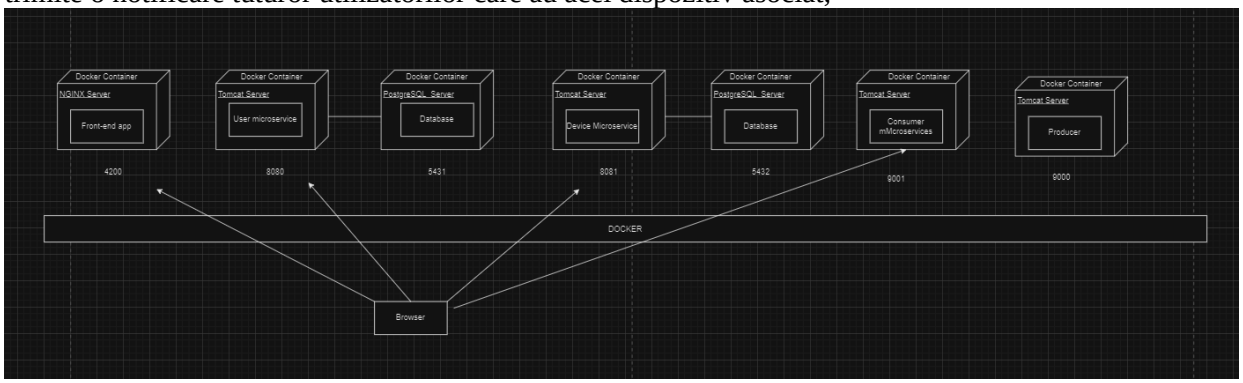
- Este responsabil cu calcularea sumei si trimiterea unei notificari in cazul in care se depaseste valoarea maxima.
- Consumul maxim pe ora va salvat in baza de date.
- Obtinerea informatiilor despre Device se obtine prin doua cozi care comunica cu Device Microservice.

Producer

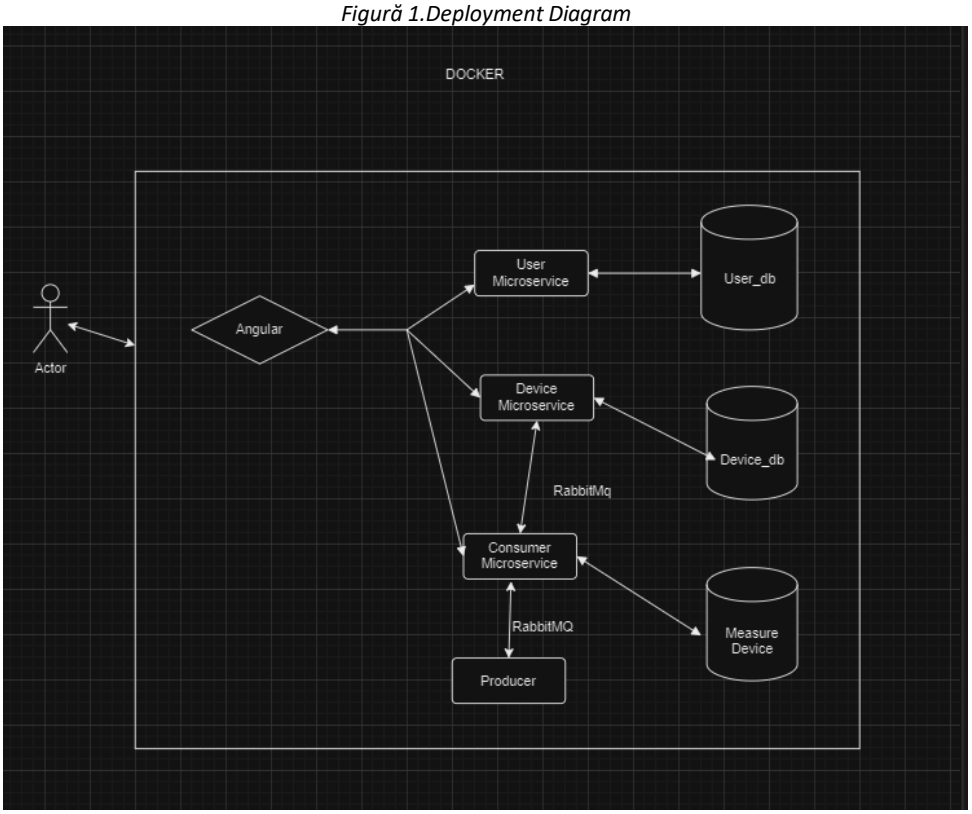
- Este responsabil pentru citirea dator din fisierul sensor.csv si trimiterea acestora printr-o coada la consumer.
- Pornirea a doua thread-uri cu ajutorul carora se vor trimite datele.

2. Implementare

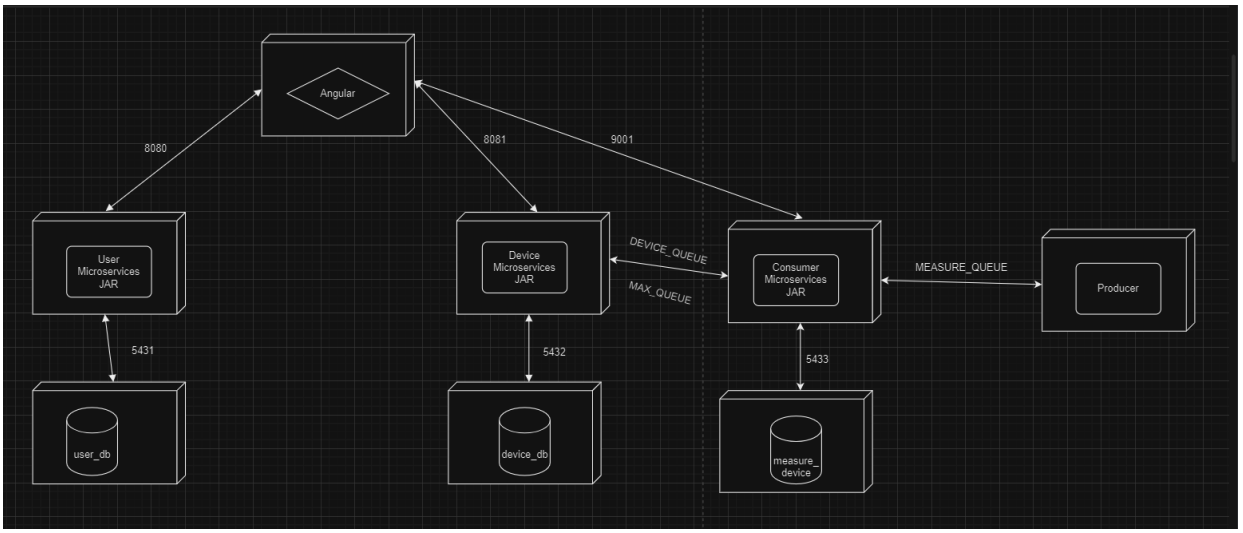
Am creat Produser-ul care se ocupa cu citirea valorilor din csv si trimiterea datelor spre Consumer, unde se primesc datele si se fac diverse operatii. In Consumer verificam existenta deviceId-ului primit de la Producer prin trimiterea acestuia prin coada „DEVICE_QUEUE”, prin primirea ca raspuns un deviceId inseamna ca id-ul primit este valid, in cazul in care raspunsul este null inseamna ca nu avem in baza de date un id cu acel numar. Dupa ce s-a facut verificarea se asteapta mesaje pana se ajunge la o anumita valoare data, cand se atinge acea valoare se calculeaza suma, se adauga in baza de date si se verifica daca a depasit valoarea maxima., caz in care se trimite o notificarea spre toti utilizatorii care au asignat acel deviceId. Valoarea maxima se obtine tot printr-o coada care comunica cu Device Microservice, prin aceasta coada se trimite un deviceId, coada din Device Microservices va primi acel deviceId, va face un get prin care va verifica daca acel id exista in baza de date si daca exista se va prelua valoarea maxima si ca fi transmise prin coada ca raspuns spre Consumer.
De fiecare data cand a trecut o ora se salveaza in baza de date hourlyEnergyConsumation alaturi de deviceId, ora si data, astfel ca in momentul in care vom crea graficele vom face un request spre un endpoint care va face un get spre tabelul in care sunt datele salvate si va lua toate valorile care au acel deviceId si se afla in data selectata din calendar. Cand se depaseste valoarea maxiam cu ajutorul WebSocket se va trimite o notificare tuturor utilizatorilor care au acel dispozitiv asociat,



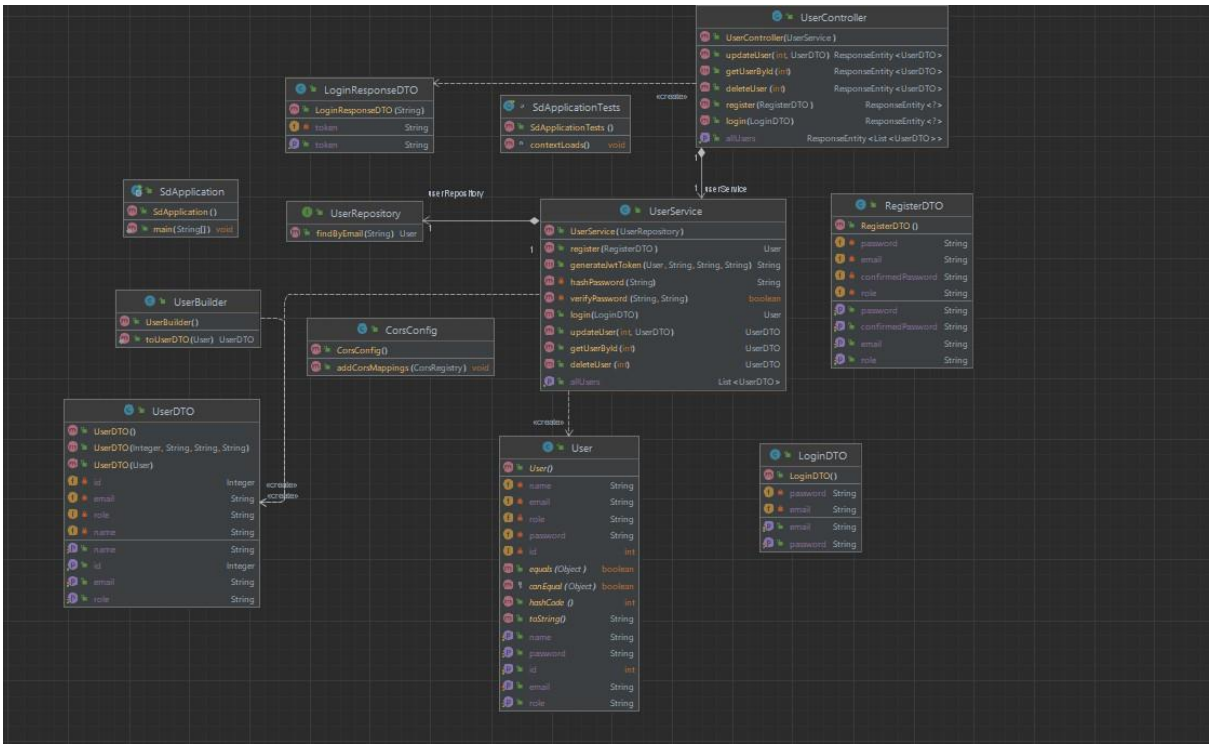
Figură 1.Deployment Diagram



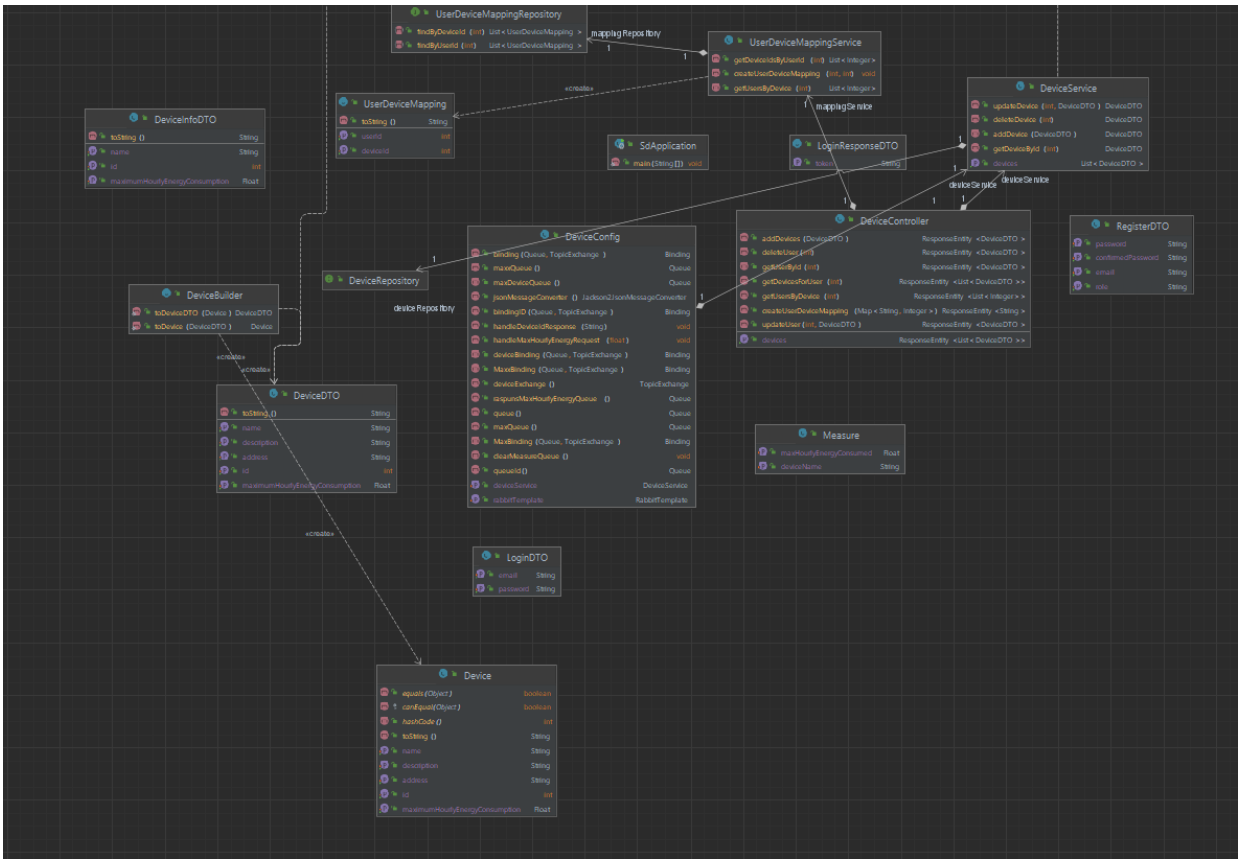
Figură 2.Conceptual Diagram



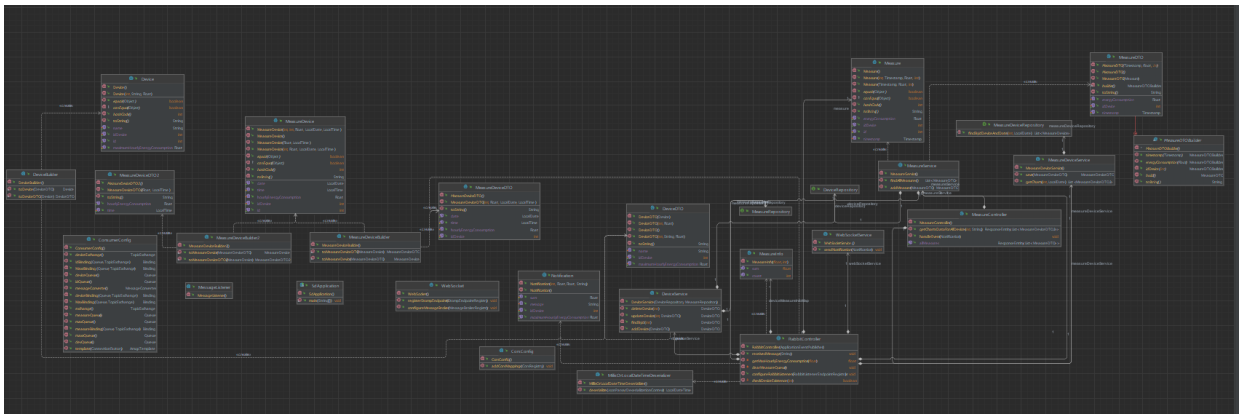
Figură 3Docker Containers



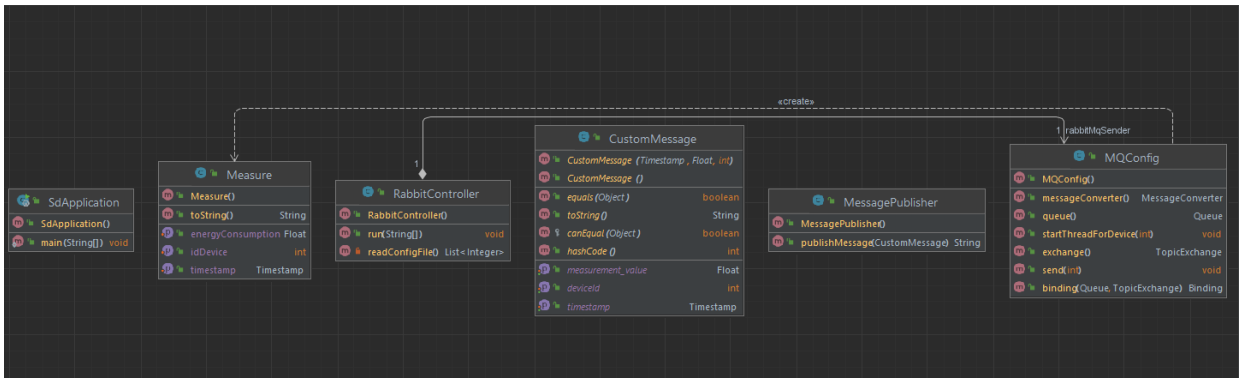
Figură 4.Diagrama UML User



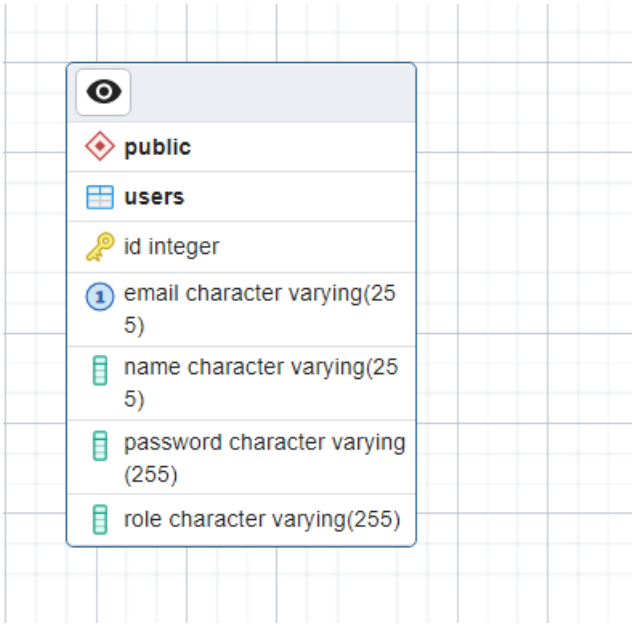
Figură 5.Diagrama UML Device



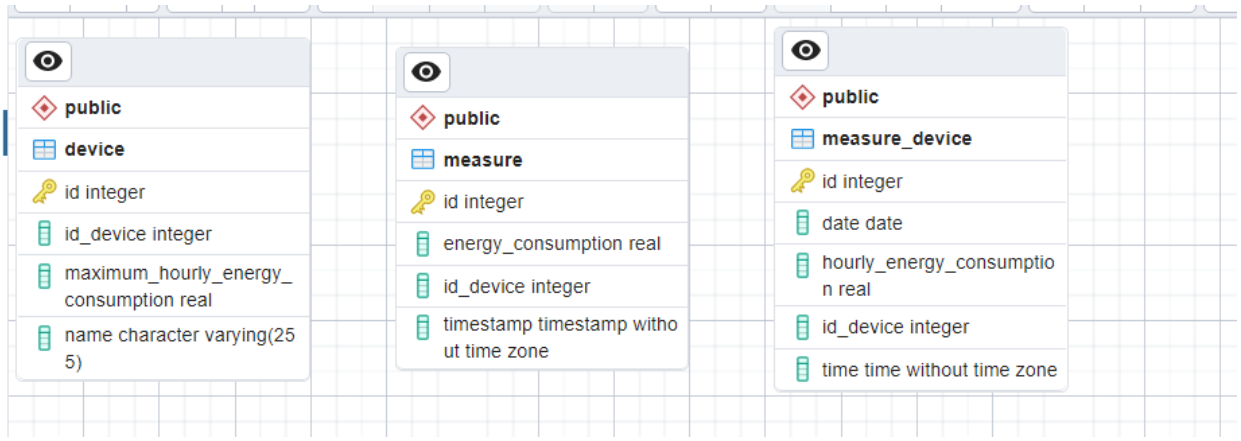
Figură 6.Diagrama UML Consumer



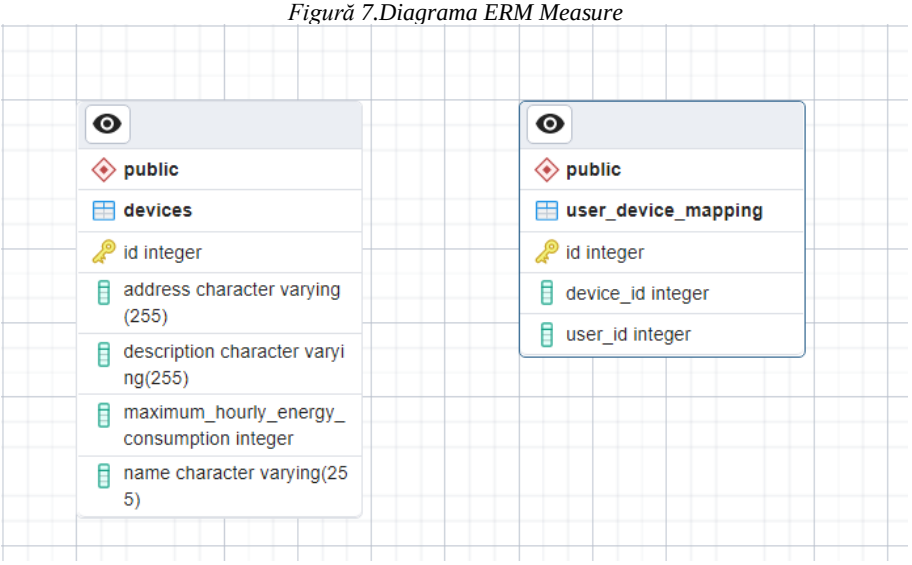
Figură 7..Diagrama UML Producer



Figură 6.Diagrama ERM User



Figură 7.Diagrama ERM Measure



Figură 7.Diagrama ERM Device

3.Read Me

Comenzi

- docker compose-up
- docker build -t frontend:latest .
- docker run --name frontend -ti -p 4200:80 frontend:latest
- docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.12-managementdocker exec -it rabbitmq /bin/bash

Considerații pentru Construire și Execuție

Frontend

Instalare și Pregătire:

Asigurați-vă că aveți Node.js și npm (Node Package Manager) instalate.

Instalați Angular CLI folosind următoarea comandă:

- npm install -g @angular/cli

Configurare și Pregătire pentru Proiect:

Navigați la directorul frontend și executați următoarele comenzi pentru a crea un proiect Angular și a instala dependențele:

- ng new energy-management-app
- cd energy-management-app
- ng generate component component_name

Pentru a rula aplicatia aveti nevoie de comanda:

- ng serve

Device, User Microservices, Producer si Consumer

Asigurați-vă că aveți Java Spring REST si un IDE precum Intellij, proiectul de spring se poate genera pe site-ul Spring initializr.

Pentru a putea comunica intre microservicii utilizand RabbitMQ, trebuie sa deschideti Docker Desktop, iar intr-un PowerShell sa rulati comanda prezentata mai sus.

3. Bibliografie

- https://start.spring.io/
- https://www.docker.com/
- https://spring.io/microservices
- https://www.npmjs.com/package/ngx-toastr
- https://www.rabbitmq.com/
- https://canvasjs.com/angular-charts/
- https://www.freecodecamp.org/news/how-to-make-bar-and-line-charts-using-chartjs-in-angular/